CONTENEURISATION AVANCÉE

Orchestration de conteneurs avec Kubernetes

Support de cours — 3ESGI (cours communs, ALL) — 5 séances de 1h30

Dr Joffrey Catena

Ce support condense le programme complet de la formation Kubernetes fournie (19 chapitres, 5 projets guidés, environ une cinquantaine d'heures de contenu dans sa version intégrale) en 5 séances de 1h30, soit 7h30 au total. Un tel volume ne permet pas de traiter tous les chapitres avec le niveau de détail d'une formation professionnelle longue : ce support priorise les dix notions fondamentales annoncées dans le programme (clusters, objets Kubernetes, services, control plane, réseau, kubectl, déploiement cloud, rollback, volumes, debugging), et relègue en ouverture de fin de parcours les sujets avancés (Helm, GitOps/Argo CD, TLS/cert-manager, ordonnancement fin, API Gateway), mentionnés sans TP dédié faute de temps. Il se lit en parallèle du workbook d'exercices (.md) : chaque module renvoie vers la phase d'exercices correspondante.

Sommaire

Séance 1 (1h30) — Introduction et architecture

- Module 1 — Qu'est-ce que Kubernetes ? Architecture et écosystème

Séance 2 (1h30) — Les objets fondamentaux

- Module 2 — Pods, Deployments, ReplicaSets et labels

Séance 3 (1h30) — Services et réseau

- Module 3 — Services, Namespaces et communication réseau

Séance 4 (1h30) — Configuration, stockage et mise à l'échelle

- Module 4 — ConfigMaps, Secrets, volumes et autoscaling

Séance 5 (1h30) — Cloud, debugging et synthèse

- Module 5 — Déploiement cloud, debugging, rollback et projet de synthèse
- Ouverture — Helm, GitOps, TLS, ordonnancement avancé

SÉANCE 1 — 1h30

Module 1 — Qu'est-ce que Kubernetes ? Architecture et écosystème

PHASE DE COURS 1 — 0 à 45 min

1.1 Pourquoi un orchestrateur de conteneurs

Docker seul permet de lancer un conteneur sur une machine, mais ne répond pas aux questions suivantes : que se passe-t-il si le conteneur plante ? Comment répartir la charge sur plusieurs machines ? Comment mettre à jour une application sans interruption de service ? Kubernetes (K8s), développé par Google en 2014 en langage Go, répond à ces questions à grande échelle : c'est aujourd'hui l'orchestrateur de conteneurs le plus utilisé au monde, parfois présenté comme le système d'exploitation du Cloud.

1.2 Architecture d'un cluster

Un cluster Kubernetes est composé de deux types de serveurs : les control planes, qui pilotent le cluster, et les nodes (ou workers), qui exécutent réellement les conteneurs.

Composant	Rôle
kube-apiserver	Point d'entrée unique de toutes les commandes (kubectl, interfaces) vers le cluster
etcd	Base de données clé-valeur stockant l'état complet du cluster
kube-scheduler	Décide sur quel node placer chaque nouveau Pod
kube-controller-manager	Fait converger l'état réel du cluster vers l'état désiré (relance un Pod manquant, etc.)
kubelet (sur chaque node)	Agent qui dialogue avec le control plane et pilote les conteneurs du node
kube-proxy (sur chaque node)	Gère les règles réseau permettant aux Services de joindre les bons Pods

À se représenter comme un schéma avec un bloc « control plane » d'un côté, ses 4 composants, et plusieurs blocs « node » de l'autre, contenant chacun kubelet, kube-proxy et des Pods. C'est ce schéma de référence qui sera repris à chaque module suivant.

1.3 L'écosystème des conteneurs : Docker, Kubernetes et OCI/CRI

Kubernetes n'exécute pas directement les conteneurs : il délègue cette tâche à un container runtime respectant les standards ouverts OCI (Open Container Initiative) via l'interface CRI (Container Runtime Interface). Docker a longtemps été le runtime le plus utilisé ; containerd (extrait du projet Docker) est aujourd'hui le runtime par défaut de la plupart des distributions Kubernetes managées. Une image Docker reste une image Docker : c'est l'outil qui l'exécute qui change.

1.4 kubectl et minikube

kubectl est le client en ligne de commande qui envoie des requêtes au kube-apiserver. minikube lance un cluster Kubernetes à un seul node sur la machine locale (dans une VM ou un conteneur), suffisant pour apprendre et tester sans infrastructure cloud.

```
minikube start                  # démarre un cluster local
kubectl cluster-info            # vérifie la connexion au cluster
kubectl get nodes               # liste les nodes du cluster
kubectl get pods -A             # liste tous les Pods, tous namespaces
minikube dashboard              # ouvre le tableau de bord web
```

> **→ Traitez la Phase d'exercices 1 du workbook (installation, premières commandes kubectl, QCM architecture).**

SÉANCE 2 — 1h30

Module 2 — Pods, Deployments, ReplicaSets et labels

PHASE DE COURS 2 — 0 à 45 min

2.1 Le Pod, plus petite unité Kubernetes

Un Pod regroupe un ou plusieurs conteneurs qui partagent le même réseau et le même stockage, et qui sont toujours déployés ensemble sur le même node. Dans la grande majorité des cas, un Pod ne contient qu'un seul conteneur ; plusieurs conteneurs dans un même Pod servent des cas précis (un conteneur principal et un conteneur annexe, dit sidecar, qui l'assiste).

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
    env: formation
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80

kubectl apply -f pod.yaml      # crée le Pod
kubectl get pods               # vérifie son état
kubectl describe pod nginx-pod # détail et événements
kubectl logs nginx-pod         # logs du conteneur
kubectl exec -it nginx-pod -- sh   # ouvre un shell dans le conteneur
```

Un Pod créé directement comme ci-dessus n'est pas auto-réparé : si on le supprime ou s'il plante, rien ne le recrée. C'est exactement le problème que les Deployments et ReplicaSets, vus juste après, viennent résoudre : en exécutant kubectl delete pod nginx-pod puis kubectl get pods, on constate qu'il n'est pas relancé.

2.2 ReplicaSet : maintenir un nombre de réplicas

Un ReplicaSet garantit qu'un nombre défini de copies (réplicas) d'un Pod tourne en permanence : si un Pod disparaît, le ReplicaSet en recrée un autre pour revenir au nombre voulu. Il s'appuie sur un sélecteur de labels pour savoir quels Pods lui appartiennent.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
      tier: frontend
  template:
    metadata:
      labels:
        app: nginx
        tier: frontend
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

> **Attention :** les labels du template (sous spec.template.metadata.labels) doivent correspondre exactement au selector.matchLabels : une erreur de frappe ici est la cause la plus fréquente d'un ReplicaSet qui ne gère aucun Pod.

2.3 Deployment : au-dessus du ReplicaSet

En pratique, on ne crée presque jamais un ReplicaSet directement : on utilise un Deployment, qui gère lui-même un ou plusieurs ReplicaSets et ajoute les mises à jour progressives (rolling update) et le retour en arrière (rollback, traité au Module 5). Changer l'image d'un Deployment crée un nouveau ReplicaSet et bascule progressivement les Pods de l'ancien vers le nouveau, sans interruption de service.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80

kubectl apply -f deployment.yaml
kubectl get deployments,replicasets,pods   # observer la hiérarchie
kubectl scale deployment nginx-deployment --replicas=5
```

2.4 Labels et sélecteurs

Les labels sont de simples paires clé/valeur attachées à un objet, sans signification imposée par Kubernetes : app, env, tier, version... Les sélecteurs (utilisés par ReplicaSet, Deployment, Service) choisissent un ensemble d'objets en fonction de leurs labels. C'est ce mécanisme, et lui seul, qui relie un Service à ses Pods, ou un Deployment à son ReplicaSet.

> **→ Traitez la Phase d'exercices 2 du workbook (écrire un Pod, un Deployment à 3 réplicas, corriger un selector mal aligné).**

SÉANCE 3 — 1h30

Module 3 — Services, Namespaces et communication réseau

PHASE DE COURS 3 — 0 à 45 min

3.1 Le problème que résout un Service

Les Pods sont éphémères : à chaque recréation, un Pod change d'adresse IP. Un Service fournit une adresse stable et un nom DNS interne qui redirigent automatiquement le trafic vers les Pods correspondant à son sélecteur, quels que soient leurs changements d'adresse.

3.2 Les types de Service

Type	Portée	Usage typique
ClusterIP (défaut)	Accessible uniquement depuis l'intérieur du cluster	Communication entre Pods (ex. backend qui appelle une base de données)
NodePort	Expose un port fixe sur chaque node du cluster	Accès externe simple, développement et tests
LoadBalancer	Provisionne un répartiteur de charge externe (chez un fournisseur cloud)	Exposition publique en production (GCP, AKS...)
ExternalName	Redirige vers un nom DNS externe au cluster	Faire pointer un nom interne vers un service externe existant

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
```

port (le port exposé par le Service) et targetPort (le port réellement écouté par le conteneur) peuvent différer : par exemple, un Service exposé en port 80 peut rediriger vers un conteneur qui écoute en interne sur le port 8080.

3.3 Namespaces

Un Namespace cloisonne logiquement un cluster en plusieurs environnements isolés (dev, staging, prod, ou un namespace par équipe), chacun avec ses propres objets. Deux objets de même nom peuvent coexister dans des Namespaces différents sans conflit.

```
kubectl create namespace formation-b3
kubectl get pods -n formation-b3
kubectl apply -f deployment.yaml -n formation-b3
```

3.4 Notions de réseau cluster

Chaque Pod reçoit sa propre adresse IP au sein d'un réseau virtuel géré par un plugin CNI (Container Network Interface). Deux Pods peuvent se joindre directement par cette IP, mais c'est le DNS interne du cluster (CoreDNS) qui permet de joindre un Service par son nom plutôt que par une adresse qui pourrait changer : un Pod du namespace formation-b3 joint le Service nginx-svc du même namespace simplement en résolvant nginx-svc, ou nginx-svc.formation-b3.svc.cluster.local depuis un autre namespace.

> **→ Traitez la Phase d'exercices 3 du workbook (Service ClusterIP et NodePort, choix du bon type selon un scénario, Namespaces).**

SÉANCE 4 — 1h30

Module 4 — ConfigMaps, Secrets, volumes et autoscaling

PHASE DE COURS 4 — 0 à 45 min

4.1 ConfigMaps : séparer la configuration du code

Un ConfigMap stocke des paires clé/valeur non sensibles (variables d'environnement, fichiers de configuration) en dehors de l'image du conteneur, pour pouvoir changer la configuration sans reconstruire l'image.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "production"
  MAX_CONNECTIONS: "100"

apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: nginx:1.27
      envFrom:
        - configMapRef:
            name: app-config
```

4.2 Secrets

Un Secret suit exactement la même logique qu'un ConfigMap, mais pour des données sensibles (mots de passe, jetons, certificats). Les valeurs y sont encodées en base64 par défaut, ce qui n'est pas un chiffrement : un Secret Kubernetes basique reste lisible par quiconque accède à l'API avec les droits suffisants. Pour une sécurité renforcée en production, on s'appuie sur un coffre externe (Secrets Vault), chiffré et avec une gestion fine des accès.

> **Attention :** bien faire comprendre aux étudiants que base64 n'est pas du chiffrement : c'est un simple encodage, réversible instantanément. Un Secret Kubernetes protège contre la lecture accidentelle en clair dans un manifeste versionné, pas contre un accès non autorisé au cluster.

4.3 Volumes : de l'éphémère au persistant

Par défaut, les données écrites par un conteneur disparaissent avec lui. Un volume éphémère (emptyDir) survit aux redémarrages de conteneurs au sein d'un même Pod, mais disparaît avec le Pod. Pour des données qui doivent survivre à la suppression du Pod, on utilise un PersistentVolume (PV, la ressource de stockage réelle) associé à une PersistentVolumeClaim (PVC, la demande de stockage faite par un Pod).

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi

apiVersion: v1
kind: Pod
```

```
metadata:
  name: mysql-pod
spec:
  containers:
    - name: mysql
      image: mysql:8.0
      volumeMounts:
        - mountPath: /var/lib/mysql
          name: mysql-storage
  volumes:
    - name: mysql-storage
      persistentVolumeClaim:
        claimName: data-pvc
```

Une StorageClass (non détaillée dans ce support) automatise la création du PV correspondant à une PVC chez un fournisseur cloud : c'est ce mécanisme qui est utilisé en pratique sur GCP ou AKS plutôt que de créer un PV à la main.

4.4 Mise à l'échelle automatique : HorizontalPodAutoscaler

Le HorizontalPodAutoscaler (HPA) ajuste automatiquement le nombre de réplicas d'un Deployment en fonction d'une métrique observée, le plus souvent l'utilisation CPU moyenne des Pods.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Ce HPA maintient entre 2 et 10 réplicas, en ajoutant des Pods dès que l'utilisation CPU moyenne dépasse 70 %. À mentionner pour la culture générale : il existe symétriquement un VerticalPodAutoscaler (ajuste les ressources allouées à chaque Pod plutôt que leur nombre) et un ClusterAutoscaler (ajuste le nombre de nodes du cluster lui-même), non détaillés ici.

> **→ Traitez la Phase d'exercices 4 du workbook (ConfigMap consommé par un Pod, PVC pour une base de données, HPA).**

SÉANCE 5 — 1h30

Module 5 — Déploiement cloud, debugging, rollback et projet de synthèse

PHASE DE COURS 5 — 0 à 45 min

5.1 Du cluster local au cluster cloud

minikube suffit pour apprendre, mais un cluster de production est hébergé chez un fournisseur cloud géré : GKE (Google), AKS (Microsoft Azure), ou DOKS (DigitalOcean) par exemple. Le principe reste identique (mêmes manifestes YAML, même kubectl), mais deux différences pratiques sont immédiates : un Service de type LoadBalancer provisionne réellement un répartiteur de charge externe avec une IP publique (alors que minikube simule cela localement), et le dimensionnement des nodes se fait depuis la console du fournisseur ou via le ClusterAutoscaler du Module 4.

```
kubectl config get-contexts        # liste les clusters connus de kubectl
kubectl config use-context mon-cluster-gke   # bascule vers un cluster cloud
kubectl get nodes -o wide          # vérifie les nodes du cluster cible
```

5.2 Rollback d'un Deployment

Chaque mise à jour d'un Deployment (changement d'image notamment) est historisée. En cas de problème après une mise à jour, on peut revenir à la version précédente sans réécrire de manifeste.

```
kubectl rollout history deployment nginx-deployment   # historique des révisions
kubectl rollout undo deployment nginx-deployment      # retour à la révision précédente
kubectl rollout undo deployment nginx-deployment --to-revision=2
kubectl rollout status deployment nginx-deployment    # suit l'avancement
```

5.3 Debugging avancé

Commande	Ce qu'elle révèle
`kubectl get pods`	État global (Running, Pending, CrashLoopBackOff...)
`kubectl describe pod <nom>`	Événements détaillés : pourquoi un Pod reste Pending ou échoue
`kubectl logs <nom> [-p]`	Logs du conteneur (-p pour ceux de l'instance précédente après un crash)
`kubectl exec -it <nom> -- sh`	Shell interactif dans le conteneur pour investiguer de l'intérieur
`kubectl get events --sort-by=.lastTimestamp`	Journal chronologique des événements du cluster ou du namespace

Une démarche de debugging méthodique part toujours du général vers le spécifique : get pods pour repérer le symptôme (statut anormal), describe pour comprendre le contexte (événements, erreurs de scheduling, images introuvables), logs pour voir ce que le conteneur a réellement produit, exec en dernier recours pour investiguer en direct. CrashLoopBackOff signifie que le conteneur démarre puis s'arrête en boucle : describe et logs sont alors les deux premiers réflexes.

5.4 Projet de synthèse guidé

Déployer une mini-application web en trois objets : un ConfigMap pour sa configuration, un Deployment avec une sonde de vitalité, et un Service pour l'exposer.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: webapp-config
data:
  SITE_NAME: "Formation B3"

apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: webapp
  labels:
    app: webapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
        - name: webapp
          image: nginx:1.27
          ports:
            - containerPort: 80
          envFrom:
            - configMapRef:
                name: webapp-config
          livenessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 5

apiVersion: v1
kind: Service
metadata:
  name: webapp-svc
spec:
  type: LoadBalancer
  selector:
    app: webapp
  ports:
    - port: 80
      targetPort: 80
```

La sonde de vitalité (livenessProbe) n'a pas été détaillée dans un module dédié : elle vérifie périodiquement que le conteneur répond, et Kubernetes le redémarre automatiquement si ce n'est plus le cas.

> **→ Traitez la Phase d'exercices 5 du workbook (diagnostiquer un CrashLoopBackOff, rollback, assemblage du projet de synthèse complet).**

Ouverture — pour la suite, au-delà de ces 5 séances

Faute de temps dans ce format de 7h30, les sujets suivants du programme complet n'ont pas été traités et constituent des prolongements naturels si d'autres séances sont dégagées :

- Helm : gestionnaire de paquets Kubernetes, pour installer et versionner des applications complexes en un seul chart
- GitOps avec Argo CD : synchronisation automatique du cluster à partir d'un dépôt Git, intégration avec des pipelines CI/CD (GitHub Actions)
- TLS et cert-manager : chiffrement HTTPS automatisé des Services exposés
- Ordonnancement avancé : affinité/anti-affinité, teintes et tolérances, priorité et préemption des Pods
- API Gateway (Traefik, Gateway API) : routage HTTP avancé au-delà d'un simple Service LoadBalancer
- Logging et monitoring : Grafana, Loki, Prometheus
- StatefulSets : gestion d'applications avec état nécessitant une identité réseau stable (bases de données répliquées notamment)